

C3 - Intelligence pour la robotique: TP5 – Navigation par perceptron

Justin Ferdinand

I INTRODUCTION

Ce TP a pour objectif de découvrir le fonctionnement du perceptron, un modèle de neurone artificiel simple, et d'en comprendre les applications à travers des exemples théoriques (modélisation d'opérateurs logiques) puis pratiques (contrôle d'un robot Thymio).

II PERCEPTRON : IMPLANTATION D'UNE PORTE LOGIQUE

Le dossier de rendu contient un notebook jupyter 'TP5_neural_model_intro.ipynb' qui implémente les codes suivants.

II.A DÉFINITION DES VARIABLES D'ENTRÉE

Les variables d'entrée du perceptron, x_1 et x_2 , prennent les valeurs 0 ou 1.

II.B FONCTION DE CALCUL DE LA SOMME PONDÉRÉE

La sortie du perceptron est déterminée par la somme pondérée suivante :

$$s = w_0 + x_1 \cdot w_1 + x_2 \cdot w_2$$

où w_0 est le biais, w_1 et w_2 les poids associés aux entrées.

II.C FONCTION D'ACTIVATION

La fonction d'activation utilisée est une fonction seuil :

$$y =$$

- 1 si $s \geq 0$,
- 0 si $s < 0$,

II.D VALIDATION SUR LES OPÉRATEURS LOGIQUES

On teste le perceptron pour modéliser les fonctions logiques OU (OR) et ET (AND) à l'aide de tables de vérité et de choix appropriés des poids.

```

1  import pandas as pd
2
3  #table de vérité pour OU (OR)
4  poids_ou = {'w0': -0.5, 'w1': 1.0,
5              'w2': 1.0}
6
7  #table de vérité pour ET (AND)
8  poids_et = {'w0': -1.5, 'w1': 1.0,
9              'w2': 1.0}
10
11 combinaisons = [(0,0), (0,1), (1,0),
12                 (1,1)]
13
14 #test pour l'opérateur OU
15 resultats_ou = []
16 for x1, x2 in combinaisons:
17     s = calcul_s(x1, x2, **poids_ou)
18     y = activation(s)
19     resultats_ou.append({'x1': x1,
20                          'x2': x2, 's': s, 'y': y})
21 df_ou = pd.DataFrame(resultats_ou)
22
23 #test pour l'opérateur ET
24 resultats_et = []
25 for x1, x2 in combinaisons:
26     s = calcul_s(x1, x2, **poids_et)
27     y = activation(s)
28     resultats_et.append({'x1': x1,
29                          'x2': x2, 's': s, 'y': y})
29 df_et = pd.DataFrame(resultats_et)
30
31 print('Table de vérité OU (OR)')

```

```
27 print(df_ou)
28 print('\nTable de vérité ET (AND)')
29 print(df_et)
```

x_1	x_2	s	y
0	0	-0.5	0
0	1	0.5	1
1	0	0.5	1
1	1	1.5	1

Table 1: Résultat du test pour l'opérateur OU (OR)

x_1	x_2	s	y
0	0	-1.5	0
0	1	-0.5	0
1	0	-0.5	0
1	1	0.5	1

Table 2: Résultat du test pour l'opérateur ET (AND)

III EXEMPLE D'APPLICATION SUR WEBOTS

Dans cette partie, on utilise un perceptron pour contrôler le robot Thymio dans Webots à partir de ses capteurs infrarouge arrière. Les valeurs des capteurs arrière gauche et arrière droit sont binarisées, puis passées en entrée d'un perceptron modélisant une porte logique ET. Si les deux capteurs détectent un obstacle, le robot avance pour s'éloigner de l'obstacle. Sinon, il s'arrête.

Pour implémenter cela, j'ai repris mon code du TP1 en modifiant le contrôle de navigation par le perceptron ET arrière.

```
1 def calcul_s(x1, x2, w0, w1, python
   w2):
2     return w0 + x1 * w1 + x2 * w2
3
4 def activation(s):
5     return 1 if s >= 0 else 0
6
7 #poids pour perceptron ET
8 poids_et = {'w0': -1.5, 'w1': 1.0,
   'w2': 1.0}
9
10 while robot.step(timestep) != -1:
11     back_left = 1 if
   prox_sensors[5].getValue() >
   THRESHOLD else 0
12     back_right = 1 if
   prox_sensors[6].getValue() >
   THRESHOLD else 0
13
14     s_et = calcul_s(back_left,
   back_right, **poids_et)
15     y_et = activation(s_et)
16     if y_et == 1:
17         #avancer si obstacle derrière
18         motor_left.setVelocity(FORWARD_SPE
19         motor_right.setVelocity(FORWARD_SP
```

```

20     print("Obstacle derrière :
      avance !")
21 else:
22     motor_left.setVelocity(0.0)
23     motor_right.setVelocity(0.0)

```

Ce code montre comment un perceptron simple peut être utilisé pour réagir à la détection d'obstacles à l'arrière du robot et générer un comportement d'évitement vers l'avant.

IV PERCEPTRON ANALOGUE

Dans cette partie, on implémente un perceptron analogue à une entrée pour contrôler la vitesse de recul du robot Thymio en fonction de la distance à un obstacle détecté par le capteur avant. La sortie du perceptron est proportionnelle à la valeur du capteur, ce qui permet d'adapter la vitesse de recul : plus l'obstacle est proche, plus le robot recule vite.

```

1                                     python
2  while robot.step(timestep) != -1:
3      front_sensor_value =
        prox_sensors[2].getValue()
4      #normalisation de la valeur du
        capteur
5      x1 = front_sensor_value / 1000.0
6      #poids du perceptron
7      w1 = 4
8      y1 = w1 * x1
9      print(f"Front sensor value:
        {front_sensor_value}, Normalized:
        {x1}, Output y1: {y1}")
10     #vitesse de recul proportionnelle à
        la sortie
11     backward_speed = min(max(0.0, y1 *
        FORWARD_SPEED), maxVelocity)
12     print(f"Calculated backward speed:
        {y1 * FORWARD_SPEED}")
13     if backward_speed > 0.05: # seuil
        pour éviter les petits bruits
14         motor_left.setVelocity(-
            backward_speed)

```

```

15     motor_right.setVelocity(-
        backward_speed)
16     print(f"Obstacle devant : recule
        à {backward_speed:.2f}")
17 else:
18     motor_left.setVelocity(FORWARD_SPE
19     motor_right.setVelocity(FORWARD_SP

```

Ce code illustre le fonctionnement d'un perceptron analogue : la sortie n'est plus binaire, mais continue, et permet un contrôle proportionnel de la vitesse du robot selon la proximité de l'obstacle détecté. Cela nous permet d'obtenir les commandes de recul suivantes :

```

Front sensor value:
1  261.2632287380115, Normalized:
   0.2612632287380115, Output y1:
   1.045052914952046
2  Calculated backward speed:
   8.360423319616368
3  Front obstacle : move backward at 8.36
Front sensor value:
4  325.08905221648115, Normalized:
   0.32508905221648116, Output y1:
   1.3003562088659246
5  Calculated backward speed:
   10.402849670927397
6  Front obstacle : move backward at 9.53
Front sensor value:
7  58.500129297333025, Normalized:
   0.058500129297333024, Output y1:
   0.2340005171893321
8  Calculated backward speed:
   1.8720041375146568
9  Front obstacle : move backward at 1.87

```

J'observe ainsi une réponse proportionnelle à la distance de l'obstacle, avec une vitesse de recul plus élevée lorsque l'obstacle est proche.

IV.A PERCEPTRON ANALOGUE À DEUX ENTRÉES

On peut implémenter le perceptron analogue à deux entrées, comme illustré dans la figure 3 du sujet. Ici, chaque entrée correspond à la valeur normalisée d'un capteur (par exemple, capteur avant gauche et capteur avant droit). Les poids w_1 et w_2 permettent de moduler l'influence de chaque capteur sur la sortie du perceptron.

Si l'on choisit w_1 beaucoup plus grand que w_2 , la sortie sera principalement influencée par le capteur associé à w_1 .

```

1  while robot.step(timestep) != -1:
2      left_sensor_value = prox_sensors[1].getValue()
3      right_sensor_value = prox_sensors[3].getValue()
4      #normalisations
5      x1 = left_sensor_value / 1000.0
6      x2 = right_sensor_value / 1000.0
7      #poids différents des capteurs
8      w1 = 5 #gauche
9      w2 = 1#droite
10
11     #perceptron à deux entrées
12     y1 = w1 * x1 + w2 * x2
13     print(f"Left sensor: {x1:.2f}, Right sensor: {x2:.2f}, Output y1: {y1:.2f}")
14     #vitesse de recul proportionnelle à la sortie
15     backward_speed = min(max(0.0, y1 * FORWARD_SPEED), maxVelocity)
16     if backward_speed > 0.05:
17         motor_left.setVelocity(-backward_speed)
18         motor_right.setVelocity(-backward_speed)
19         print(f"Obstacle devant : recule à {backward_speed:.2f}")
20     else:

```

```

21         motor_left.setVelocity(FORWARD_SPE
22         motor_right.setVelocity(FORWARD_SP

```

Ce code montre que le capteur associé au poids le plus fort a une influence dominante sur la sortie du perceptron et donc sur la vitesse de recul du robot. En modifiant les valeurs de w_1 et w_2 , on peut observer expérimentalement l'effet de chaque capteur sur le comportement du robot dans le même scénario :

Avec capteur gauche plus influent ($w_1 = 5$, $w_2 = 1$) :

```

1  Left sensor: 0.00, Right sensor: 0.00, Output y1: 0.00
2  Left sensor: 0.31, Right sensor: 0.00, Output y1: 1.57
3  Front obstacle : move backward at 9.53

```

Le capteur gauche influe fortement sur la sortie, entraînant un recul rapide car le mur gauche est proche.

Avec capteur droit plus influent ($w_1 = 1$, $w_2 = 5$) :

```

1  Left sensor: 0.01, Right sensor: 0.00, Output y1: 0.01
2  Front obstacle : move backward at 0.09

```

Ici, le capteur droit ayant peu d'influence et ne rencontrant pas d'obstacle, la sortie reste faible mais positive car toujours influencée par le capteur gauche.

V VÉHICULE DE BRATEINBERG

V.A PRINCIPE DE L'ALGORITHME

Le robot utilise ses capteurs avant (gauche, centre, droite) pour détecter les obstacles et décide de tourner ou d'avancer selon la sortie du perceptron :

- Si obstacle devant, il tourne sur lui-même.
- Si obstacle à gauche, il tourne à droite.
- Si obstacle à droite, il tourne à gauche.
- Sinon, il avance.

De plus, si les deux capteurs arrière détectent un obstacle, le robot avance pour éviter de rester bloqué. Les rayons infrarouges de Thymio sont visibles sur la Figure 1.

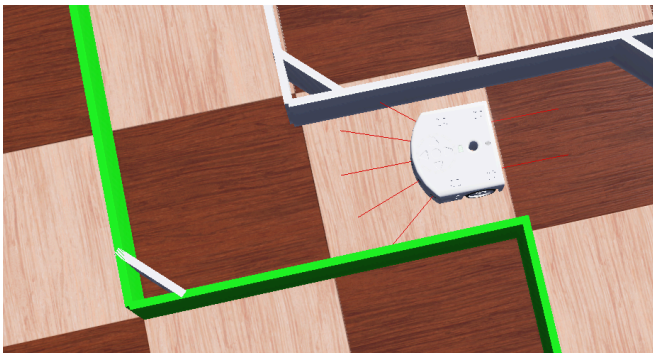


Figure 1: Capture d'écran du robot Thymio avec les capteurs infrarouge

V.B EXTRAIT DE CODE

Le code suivant implémente le modèle de la Figure 2 pour la navigation autonome du robot Thymio. Les capteurs avant sont normalisés et utilisés comme entrées d'un réseau de neurones simple: la sortie y_1 contrôle l'avance/recul, y_2 la rotation.

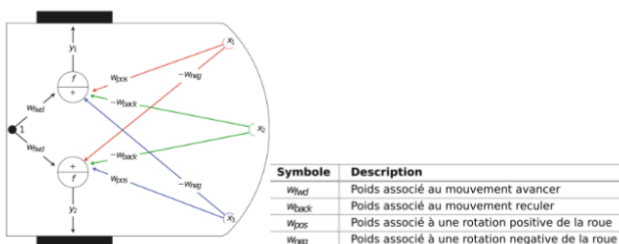


Figure 2: Modèle d'ANN proposé

Explication des poids:

- W_{fwd} : favorise l'avancée du robot en l'absence d'obstacle.
- W_{back} : s'oppose à l'avancée si un obstacle est détecté devant. Plus W_{back} est grand, plus le robot recule de manière rapide.
- W_{pos} : fait tourner le robot à droite si un obstacle est détecté à gauche.
- W_{neg} : fait tourner le robot à gauche si un obstacle est détecté à droite.
- Il n'y a pas de relation directe entre W_{back} et W_{pos} ainsi qu'entre W_{back} et W_{neg} , mais il faut que W_{back} soit suffisamment grand pour que le robot recule fortement en cas d'obstacle devant, sans empêcher la rotation.
- W_{pos} et W_{neg} doivent être inverses et équilibrés pour permettre une rotation efficace.

Pour une stratégie d'évitement d'obstacle efficace, je propose les valeurs suivantes :

- $W_{fwd} = 0.5$,
- $W_{back} = -1.0$
- $W_{pos} = 1.0$
- $W_{neg} = -1.0$

Équations de calcul:

Les sorties du réseau sont calculées ainsi:

$$y_1 = W_{fwd} \cdot 1 + W_{back} \cdot \text{center} \quad 1.$$

$$y_2 = W_{pos} \cdot \text{left} + W_{neg} \cdot \text{right} \quad 2.$$

où left, center et right sont les valeurs normalisées des capteurs avant gauche, centre et droite.

Après application d'une fonction d'activation non-linéaire, les vitesses des roues sont données par :

$$v_{\text{left}} = V_{fwd} \cdot y_1 - V_{\text{turn}} \cdot y_2 \quad 3.$$

$$v_{\text{right}} = V_{fwd} \cdot y_1 + V_{\text{turn}} \cdot y_2 \quad 4.$$

où V_{fwd} est la vitesse pour avancer et V_{turn} la vitesse de rotation.

Voici l'implémentation python du perceptron pour la navigation autonome :

```

1 python
2 FORWARD_SPEED = 8.0
3 TURN_SPEED = 4.0
4 MAX_SENSOR = 4000.0 #https://www.
  cyberbotics.com/doc/guide/thymio2?
  version=R2019a-rev1
5
6 # Entrées : [1, gauche, centre,
  droite]
7 # Sorties : y1 (avance/recule), y2
  (rotation)
8 W_fwd = 0.5 # avancer
9 W_back = -1.0 # reculer
10 W_pos = 1.0 # tourner à droite
11 W_neg = -1.0 # tourner à gauche
12
13 def activation(x):
14     #activation linéaire
15     return math.tanh(x)
16
17 while robot.step(timestep) != -1:
18     #normalisation
19     left =
      prox_sensors[4].getValue() /
      MAX_SENSOR
20     center =
      prox_sensors[2].getValue() /
      MAX_SENSOR
21     right =
      prox_sensors[0].getValue() /
      MAX_SENSOR
22     bias = 1.0
23
24     x = [bias, left, center, right]
25
26     # Calcul des sorties du réseau
27     # y1 = sortie avance/recule
28     # y2 = sortie rotation
29
30     #W_back > W_fwd car on veut
      reculer si un obstacle est devant

```

```

31 y1 = W_fwd * x[0] + W_back * x[2]
32
33
34 #si obstacle à gauche -> tourner
  à droite.
35 #si obstacle à droite -> tourner
  à gauche
36 y2 = W_pos * x[1] + W_neg * x[3]
37
38 #sorties activées
39 y1 = activation(y1)
40 y2 = activation(y2)
41
42 #vitesse des roues
43 v_left = FORWARD_SPEED * y1 -
  TURN_SPEED * y2
44 v_right = FORWARD_SPEED * y1 +
  TURN_SPEED * y2
45
46 #vitesse plafonnée
47 v_left = max(-FORWARD_SPEED,
  min(FORWARD_SPEED, v_left))
48 v_right = max(-FORWARD_SPEED,
  min(FORWARD_SPEED, v_right))
49
50 motor_left.setVelocity(v_left)
51 motor_right.setVelocity(v_right)
52
53 print(f"Capteurs (norm):
  L={left:.2f} C={center:.2f}
  R={right:.2f} | y1={y1:.2f}
  y2={y2:.2f} | vL={v_left:.2f}
  vR={v_right:.2f}")

```

Lorsque Thymio reçoit une forte contribution négative sur y_1 (recul) et une contribution positive sur y_2 (rotation), il recule et tourne pour éviter l'obstacle détecté par le capteur central.

Je remarque alors pendant la simulation que le robot thymio réagit de manière fluide à la présence des murs dans le parcours. Il est capable de faire un tour complet selon la simulation mais, puisque son raisonnement reste naïf, il ne lui permet pas de comprendre quelle est la direction à prendre. Il lui

arrive ainsi de faire demi-tour et repartir en sens inverse. Parfois, il peut rester bloqué dans un coin si les capteurs détectent des obstacles de manière continue.

Deux vidéos démontrent la capacité de navigation autonome naïve, l'une dans un environnement sans obstacle, l'autre avec.

VI SUIVI D'OBJETS

Dans cette partie, on souhaite que le robot suive un objet selon les règles suivantes :

- Le robot avance en l'absence d'objet.
- Il s'arrête si le capteur central détecte un objet très proche.
- Il tourne à gauche si un objet est détecté à gauche.
- Il tourne à droite si un objet est détecté à droite.

Pour cela, on utilise un perceptron dont la sortie y_1 contrôle l'avance/arrêt et y_2 la rotation. Les poids sont choisis pour obtenir le comportement souhaité.

Réponses aux questions sur le suivi d'objet:

1. Relation entre w_{fwd} et w_{stop} :

w_{fwd} favorise l'avance, w_{stop} s'oppose à l'avancée si un objet est détecté devant. Pour que le robot s'arrête en présence d'un objet proche, il faut que w_{stop} soit suffisamment grand par rapport à w_{fwd} .

2. Relation entre w_{fwd} et w_{pos} ainsi qu'entre w_{fwd} et w_{neg} :

w_{fwd} agit sur l'avance/arrêt, w_{pos} et w_{neg} sur la rotation. Ils doivent être choisis pour que le robot puisse tourner tout en avançant.

3. Relation entre w_{stop} et w_{pos} ainsi qu'entre w_{stop} et w_{neg} :

De la même manière que le véhicule de brateinberg, il n'y a pas de relation directe entre w_{stop} et w_{pos} ainsi qu'entre w_{stop} et w_{neg} . Seul w_{stop} doit être assez grand pour arrêter le robot.

4. Relation à respecter entre w_{pos} et w_{neg} :

w_{pos} et w_{neg} doivent être de signes opposés et de même valeur pour que le robot tourne symétriquement à gauche ou à droite selon la détection.

5. Poids pour une stratégie de suivi d'objet:

- w_{fwd} : 1.0 (avance)
- w_{stop} : -6.0 (arrêt si objet devant)
- w_{pos} : -2.0 (tourne à gauche si objet à gauche)
- w_{neg} : 2.0 (tourne à droite si objet à droite)

6. Comportement si le capteur gauche et le capteur central détectent un obstacle simultanément:

Le robot reçoit une faible valeur de y_1 (0), et une contribution négative sur y_2 (tourne à gauche). Il aura tendance à s'arrêter ou à tourner sur place vers la gauche.

Code simplifié du perceptron pour le suivi d'objet:

```
1 FORWARD_SPEED = 8.0
2 TURN_SPEED = 4.0
3 MAX_SENSOR = 4000.0
4 W_fwd = 1.0
5 W_stop = -6.0
6 W_pos = -2.0
7 W_neg = 2.0
8 SEUIL_STOP = 0.6
9
10 while robot.step(timestep) != -1:
11     left =
12     prox_sensors[4].getValue() /
13     MAX_SENSOR
14     center =
15     prox_sensors[2].getValue() /
16     MAX_SENSOR
17     right =
18     prox_sensors[0].getValue() /
19     MAX_SENSOR
20     y1 = W_fwd * 1.0 + W_stop * center
21     y2 = W_pos * left + W_neg * right
```

```
16  y1 = activation(y1)
17  y2 = activation(y2)
18
19  if center > SEUIL_STOP:
20      v_left = 0.0
21      v_right = 0.0
22  else:
23      v_left = FORWARD_SPEED * y1 -
        TURN_SPEED * y2
24      v_right = FORWARD_SPEED * y1 +
        TURN_SPEED * y2
25
26  v_left = max(-FORWARD_SPEED,
        min(FORWARD_SPEED, v_left))
27  v_right = max(-FORWARD_SPEED,
        min(FORWARD_SPEED, v_right))
28  motor_left.setVelocity(v_left)
29  motor_right.setVelocity(v_right)
```

En plaçant un objet devant Thymio, je remarque que celui-ci se déplace en suivant l'objet. Sa trajectoire s'ajuste à la position. Si je place l'objet devant le robot, ce dernier s'arrête. La vidéo démontrant le suivi d'objet est disponible dans le dossier du TP.

VII CONCLUSION

Pour conclure, ce TP m'a permis de comprendre le fonctionnement des perceptrons, tant binaires qu'analogiques, et leur application au contrôle de robots. J'ai pu expérimenter avec différents poids et observer comment ils influencent le comportement du robot Thymio dans Webots. L'implémentation d'un modèle de véhicule de Braitenberg m'a également aidé à saisir comment des réseaux de neurones simples peuvent générer des comportements complexes à partir de signaux sensoriels. Ce TP a renforcé mes compétences en programmation robotique et en intelligence artificielle appliquée à la robotique.